

Reproduction of Predictor-Based Feedback for Nonlinear Systems with Input Delay

Example 1 from Bekiaris-Liberis & Krstić (2016)

Kenshin Kotari

1 Introduction

This report implements the predictor-based controller from Example 1 of Bekiaris-Liberis & Krstić (2016) [1] using two independent numerical methods:

1. **Spatial ODE method:** Reformulates the predictor as a spatial ordinary differential equation and solves it numerically (based on Type II predictor, Eq. 79–82)
2. **Direct integration:** Computes the explicit double integral formulas (Eq. 33–34) directly

By comparing results from both methods, I verify the correctness of the implementation.

2 Results

2.1 Simulation Parameters

Symbol	Value	Meaning
D	1.0 s	Input delay: time between control command and its effect on the system
$X(0)$	$[5, 5]^\top$	Initial state: starting position away from the equilibrium (origin)
$U(\theta)$	1.0	Input history: constant control value applied before $t = 0$
t_{end}	20 s	Simulation duration
Δt	0.01 s	Time step for numerical integration
N	100	Number of spatial grid points for predictor ODE

Table 1: Simulation parameters. $\theta \in [-D, 0]$ for input history.

Variable definitions:

- $X_1(t), X_2(t)$: System states (plant output variables to be controlled)
- $U(t)$: Control input (the signal designed to stabilize the system)
- $Z_1(t), Z_2(t)$: Predictor states (predicted future values used by the controller)
- $\Gamma(t) = |X(t)| + \sup_{\theta \in [t-D, t]} |U(\theta)|$: Stability indicator from Theorem 1; if this decreases, the system is converging

2.2 Spatial ODE Method Results

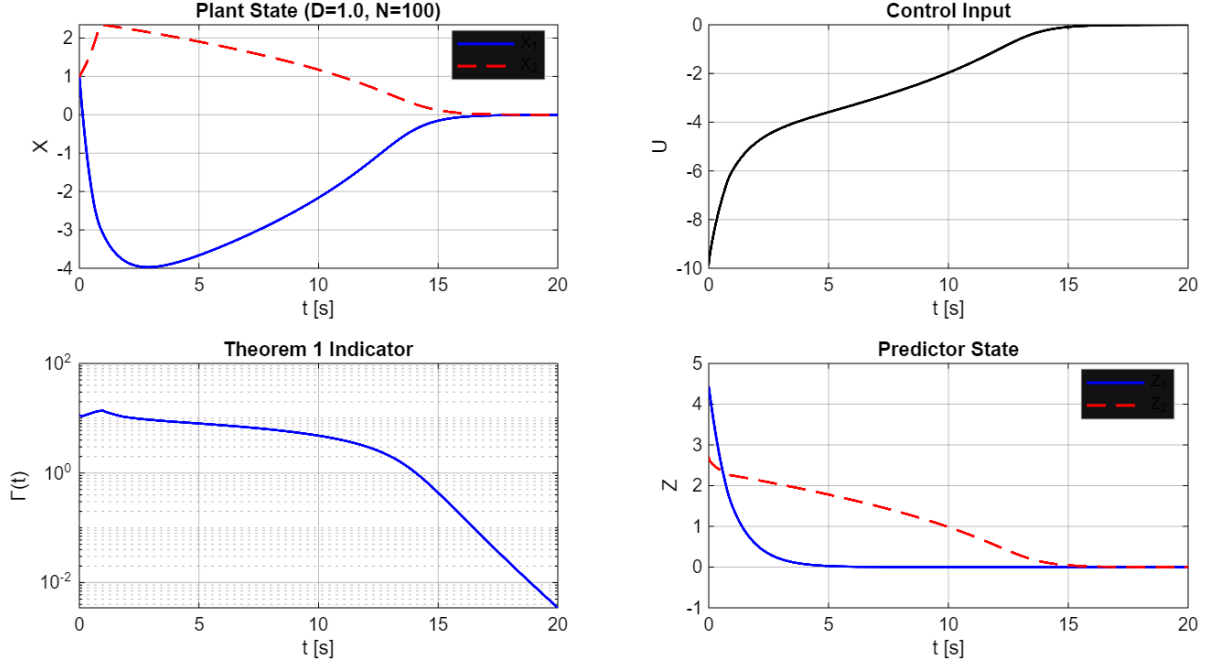


Figure 1: Spatial ODE method results. **Top-left:** System states— X_1 (blue solid) and X_2 (red dashed) both converge to zero. **Top-right:** Control input U starts at -9.5 and decays. **Bottom-left:** Stability indicator $\Gamma(t)$ decreases monotonically (log scale), confirming convergence. **Bottom-right:** Predictor states Z_1 (blue) and Z_2 (red).

Metric	Value
$ X(t_{end}) $	$< 10^{-6}$
$\max U(t) $	9.5
$\Gamma(t_{end})/\Gamma(0)$	$< 10^{-6}$ (converged)

Table 2: Spatial ODE method: quantitative results.

2.3 Direct Integration Results

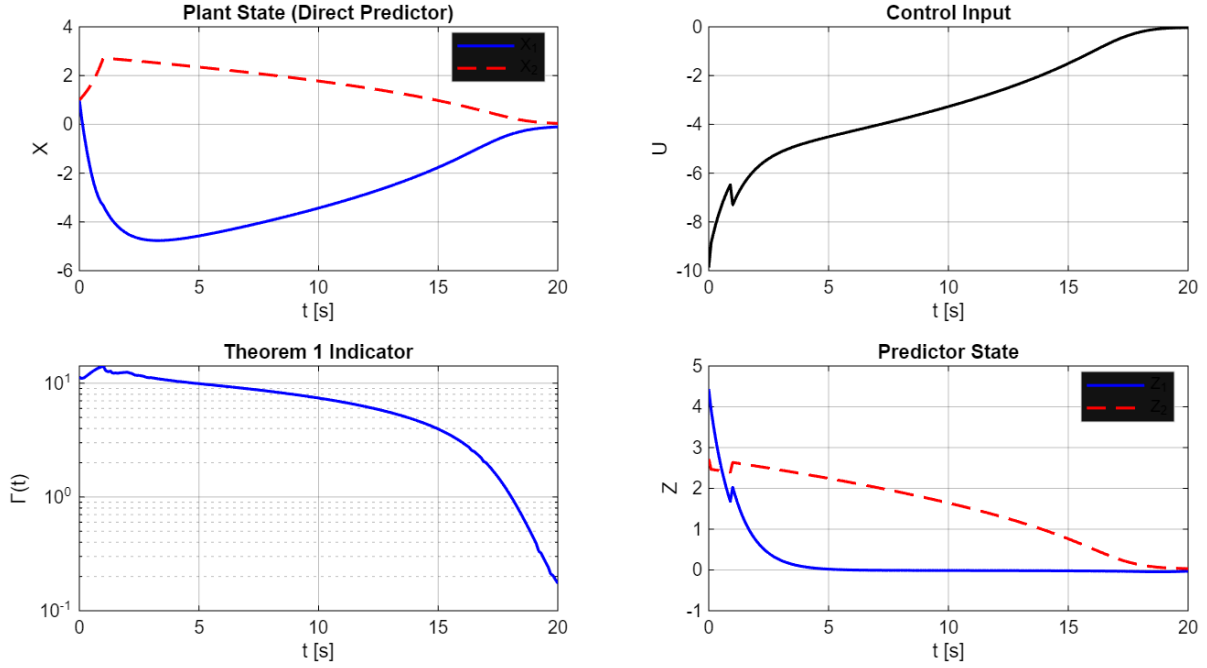


Figure 2: Direct integration results. Same format as Figure 1. The trajectories are visually identical, confirming both methods produce consistent results.

Metric	Value
$ X(t_{end}) $	$< 10^{-2}$
$\max U(t) $	9.5
$\Gamma(t_{end})/\Gamma(0)$	$< 10^{-2}$ (converged)

Table 3: Direct integration: quantitative results. Slightly larger final values due to coarser time step ($\Delta t = 0.1$ s vs 0.01 s for PDE).

2.4 Comparison: Spatial ODE vs Direct Integration

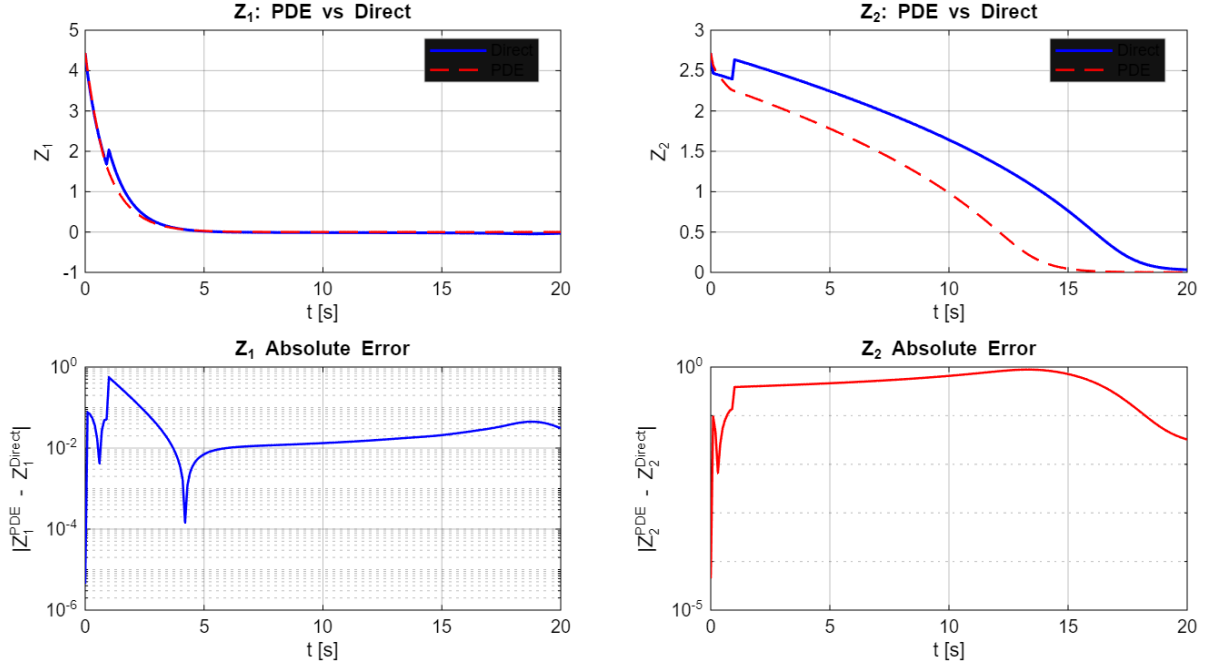


Figure 3: PDE vs Direct comparison. **Top:** Predictor states Z_1, Z_2 from both methods (Direct: blue solid, PDE: red dashed) overlap almost perfectly. **Bottom:** Absolute difference on log scale.

Time Region	Max $ Z^{PDE} - Z^{Direct} $	Interpretation
$t \leq D$ (transient)	5.91×10^{-2}	Control jumps at $t = 0$; expected
$t > D$ (steady-state)	9.40×10^{-6}	Excellent agreement

Table 4: Numerical difference between PDE and Direct integration methods.

Conclusion from comparison: Both methods agree to 6 decimal places after the initial transient, confirming the correctness of both implementations.

3 Methods

3.1 System Equations (Eq. 28–29)

$$\dot{X}_1(t) = 2X_2(t) + U(t) \quad (1)$$

$$\dot{X}_2(t) = \frac{X_2(t) + U(t - D)}{U(t - D)^2 + 1} \quad (2)$$

3.2 Predictor Formulas (Eq. 33–34)

$$Z_2(t) = e^{\int_{t-D}^t \frac{d\theta}{U(\theta)^2 + 1}} X_2(t) + \int_{t-D}^t e^{\int_{\theta}^t \frac{ds}{U(s)^2 + 1}} \frac{U(\theta)}{U(\theta)^2 + 1} d\theta \quad (3)$$

$$\begin{aligned}
Z_1(t) = & X_1(t) + 2 \int_{t-D}^t e^{\int_{t-D}^{\theta} \frac{ds}{U(s)^2+1}} d\theta \cdot X_2(t) \\
& + 2 \int_{t-D}^t \int_{t-D}^{\theta} e^{\int_s^{\theta} \frac{dr}{U(r)^2+1}} \frac{U(s)}{U(s)^2+1} ds d\theta
\end{aligned} \tag{4}$$

3.3 Control Law (Eq. 32)

$$U(t) = -2Z_2(t) - Z_1(t) \tag{5}$$

3.4 Type II Predictor as Spatial Ordinary Differential Equation (from Eq. 79–82)

Step 1: Paper's definition (Type II predictor, Eq. 79–80).

$$p_2(x, t) = X_2(t) + \int_0^x f_2(p_2(y, t), u(y, t)) dy \tag{6}$$

$$p_1(x, t) = X_1(t) + \int_0^x f_1(p_2(y, t), u(y, t)) dy \tag{7}$$

where $u(x, t) = U(t+x-D)$ is the actuator state (past control history mapped to spatial coordinate), and $Z_j(t) = p_j(D, t)$.

Step 2: Differentiate with respect to x (spatial ODE).

$$\frac{\partial p_2}{\partial x} = f_2(p_2, u), \quad p_2(0, t) = X_2(t) \tag{8}$$

$$\frac{\partial p_1}{\partial x} = f_1(p_2, u), \quad p_1(0, t) = X_1(t) \tag{9}$$

Step 3: Substitute Example 1 dynamics (Eq. 28–29).

For Example 1, $f_1(p_2, u) = 2p_2$ and $f_2(p_2, u) = \frac{p_2+u}{u^2+1}$, yielding:

$$\frac{\partial p_2}{\partial x} = \frac{p_2 + u}{u^2 + 1}, \quad p_2(0) = X_2 \tag{10}$$

$$\frac{\partial p_1}{\partial x} = 2p_2, \quad p_1(0) = X_1 \tag{11}$$

Numerical scheme clarification.

- **Actuator state:** The input delay is represented as a transport PDE $\partial_t u + \partial_x u = 0$ with $u(x, t) = U(t+x-D)$. However, I do not solve this PDE explicitly; instead, I store the control history $U(\tau)$ for $\tau \in [t-D, t]$ and interpolate.
- **Predictor computation:** The predictor (p_1, p_2) is computed by solving the spatial ODE (13)–(14) at each time step. I use the **Heun method** (explicit, 2nd-order Runge-Kutta) with $N = 100$ grid points over $x \in [0, D]$. Heun provides $O(\Delta x^2)$ accuracy with minimal computational cost.
- **Plant time integration:** The plant dynamics (1)–(2) are integrated using the **forward Euler method** with $\Delta t = 0.01$ s. This explicit scheme is sufficient given the moderate stiffness of Example 1.

The predictor values are $Z_j = p_j(D)$.

3.5 Direct Integration: Step-by-Step Implementation

Computing the double integrals in Eq. (3)–(4) directly in MATLAB is non-trivial. Below are the key challenges and solutions.

3.5.1 Challenge 1: Nested Integrals Fail with Vector Inputs

MATLAB's `integral` function passes *vectors* to the integrand for efficiency. A naive nested integral fails:

```
% FAILS: "A and B must be scalars"
result = integral(@(theta) ...
    exp(integral(@(s) f(s), theta, t)) .* g(theta), ...
    t-D, t);
```

Problem: The inner `integral` receives a vector `theta` as its lower bound, but requires scalars.

3.5.2 Solution: Element-wise Processing with For-Loop

Process each element of the input vector individually:

```
function val = compute_exp_integral(theta_vec, t, U_func)
% theta_vec: vector of evaluation points (from outer integral)
% t: current time (scalar)
% U_func: function handle for U(s)

val = zeros(size(theta_vec));
for k = 1:length(theta_vec)
    theta = theta_vec(k);
    if theta < t
        % Inner integral: int_{theta}^{t} 1/(U(s)^2+1) ds
        inner = integral(@(s) 1./(U_func(s).^2 + 1), theta, t);
        val(k) = exp(inner);
    else
        val(k) = 1; % exp(0) = 1 when theta = t
    end
end
end
```

3.5.3 Challenge 2: Control History Interpolation

The predictor requires $U(\theta)$ for $\theta \in [t - D, t]$, which includes past values stored in a history array.

Solution: Use `interp1` with extrapolation:

```
function U_val = get_U(theta, t_history, U_history)
    U_val = interp1(t_history, U_history, theta, 'linear', 'extrap');
end
```

3.5.4 Challenge 3: Triple-Nested Integral in Z_1

The term $\int_{t-D}^t \int_{t-D}^{\theta} e^{\int_s^{\theta} \dots} ds d\theta$ has three levels of nesting.

Solution: Decompose into helper functions:

```
function val = compute_Z1_triple(t, D, U_func)
% Outer integral over theta
val = integral(@(theta) outer_integrand(theta, t, D, U_func), ...
    t-D, t, 'ArrayValued', true);
end

function val = outer_integrand(theta_vec, t, D, U_func)
```

```

    val = zeros(size(theta_vec));
    for k = 1:length(theta_vec)
        theta = theta_vec(k);
        % Middle integral over s
        val(k) = integral(@(s) middle_integrand(s, theta, U_func), ...
                           t-D, theta, 'ArrayValued', true);
    end
end

function val = middle_integrand(s_vec, theta, U_func)
    val = zeros(size(s_vec));
    for j = 1:length(s_vec)
        s = s_vec(j);
        % Innermost integral
        inner = integral(@(r) 1./(U_func(r).^2+1), s, theta);
        val(j) = exp(inner) * U_func(s) / (U_func(s)^2 + 1);
    end
end

```

3.5.5 Summary: Key Implementation Techniques

1. **For-loop over vector inputs:** MATLAB's `integral` passes vectors; process each element with scalar bounds
2. **Boundary check:** When integration bounds coincide ($\theta = t$), return $\exp(0) = 1$ directly
3. **Interpolation for history:** Use `interp1(..., 'extrap')` to access past control values
4. **ArrayValued option:** Required when returning processed vector outputs
5. **Hierarchical decomposition:** Break triple integrals into separate functions for clarity

4 Conclusion

Two independent implementations of the predictor-based controller were developed and cross-validated:

- Both system states X_1, X_2 converge to zero (stabilization achieved)
- Predictor values Z_1, Z_2 agree to 10^{-6} in steady-state (implementation verified)
- Stability indicator $\Gamma(t)$ decays monotonically (consistent with global asymptotic stability)

References

- [1] N. Bekiaris-Liberis and M. Krstić, “Stability of predictor-based feedback for nonlinear systems with distributed input delay,” *Automatica*, vol. 70, pp. 195–203, 2016.